Facultad de Postgrado

Maestría en Gestión de Tecnologías de la Información

Arquitectura de Sistemas de Información

Laboratorio #03

Jerarquía de Memoria

Presentado por: Armando Lozano Bustillo

Cta. 12613070

Asesorado por:

Dr. Servio Palacios

Tegucigalpa, Honduras, 16 de Marzo de 2026

Enlaces Github

https://github.com/alozanob201/Armando_Lozano

CONTENIDO

Introducción 3
Desarrollo 4
Parte 1: Exploración de la Arquitectura del Sistema 5
Resultado de la Verificación: 6
Parte 2: Benchmark de Latencia de Memoria 7
Tabla de resultados 12
Parte 3: Visualización de la Jerarquía de Memoria 13
Preguntas y respuestas: 13
Parte 4: Experimento de Comportamiento de Cache 15
Código del Programa: 15
Procedimiento de ceración y ejecución en Linux: 17
Verificación Visual en la máquina Virtual: 17
Tabla de Resultados: 18
Preguntas y Respuestas: 18
Parte 5: Implicaciones de Seguridad 20
Relación con el Laboratorio en el ámbito Practico: 21
Conclusiones 22
Referencias 23

Introducción

El presente laboratorio se desarrolló con objetivo analizar de forma práctica el comportamiento de la jerarquía de memoria en un sistema moderno, observando cómo los distintos niveles de caché y la memoria principal influyen directamente en el rendimiento. Para ello, se trabajó en un entorno Linux sobre una máquina virtual, realizando primero una exploración de la arquitectura del sistema para identificar características como núcleos del procesador, memoria disponible, tamaño de cachés y arquitectura utilizada.

Posteriormente, se ejecutaron pruebas de latencia con herramientas de benchmark y se representaron los resultados mediante una gráfica, con el fin de visualizar cómo cambia el tiempo de acceso a medida que aumenta el tamaño del conjunto de trabajo. Finalmente, se implementó un programa en C para comparar patrones de acceso secuencial y aleatorio, permitiendo comprobar en la práctica la importancia de la localidad de referencia y del uso eficiente de la caché. En conjunto, el laboratorio permitió relacionar conceptos teóricos de arquitectura de sistemas con resultados observables en la ejecución real.

Desarrollo

Preparación de la máquina virtual para el desarrollo del laboratorio:

Para este laboratorio se utilizó una máquina virtual con sistema operativo Linux Ubuntu 24.04.2, ejecutada sobre un entorno de virtualización KVM. La máquina contó con arquitectura x86_64 de 64 bits, 4 núcleos de CPU, aproximadamente 7.8 GiB de memoria RAM.

Ilustración 1 Creación y Asignación de Recursos de VM

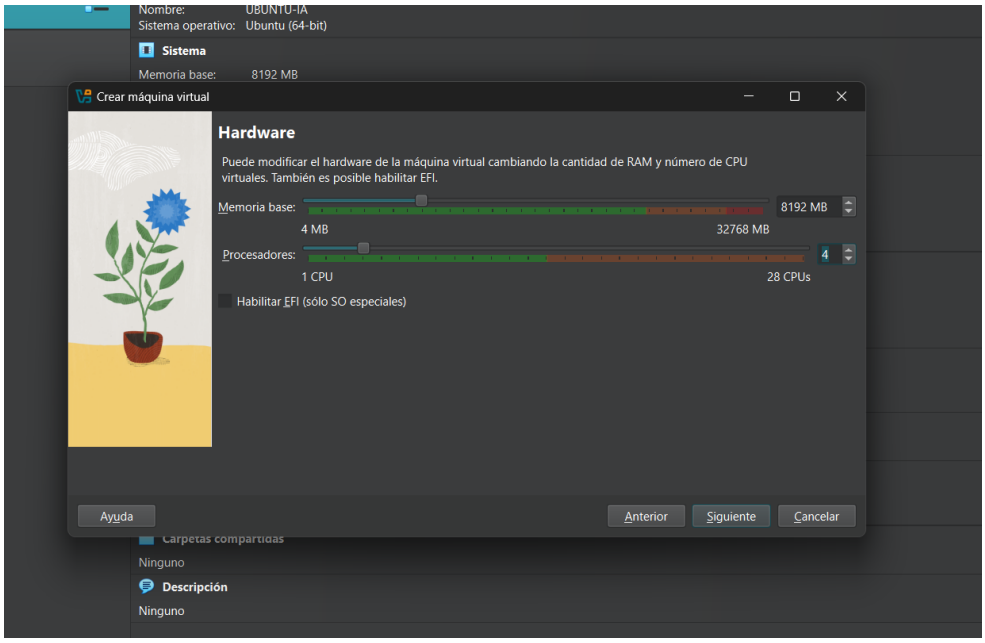
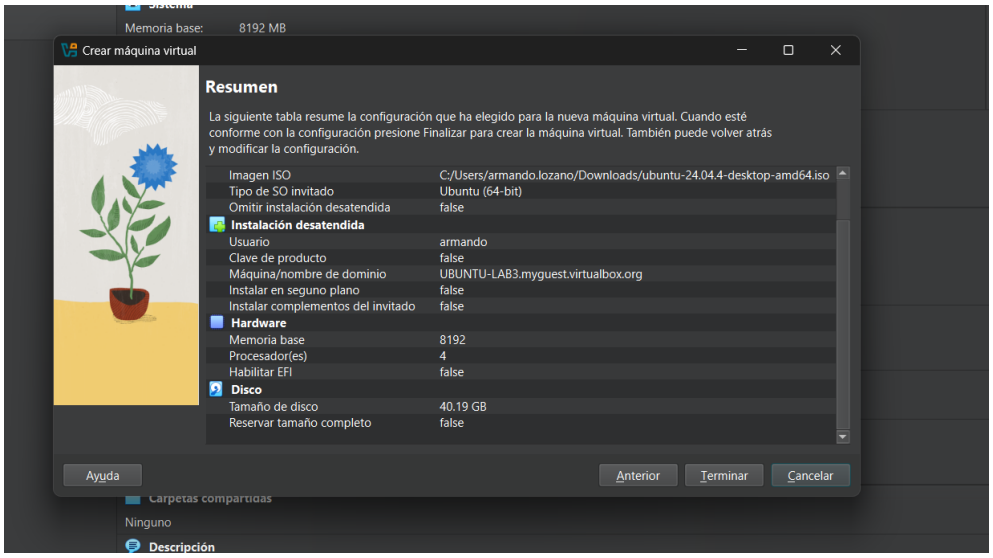


Ilustración 2 Resumen de los Elementos de la VM

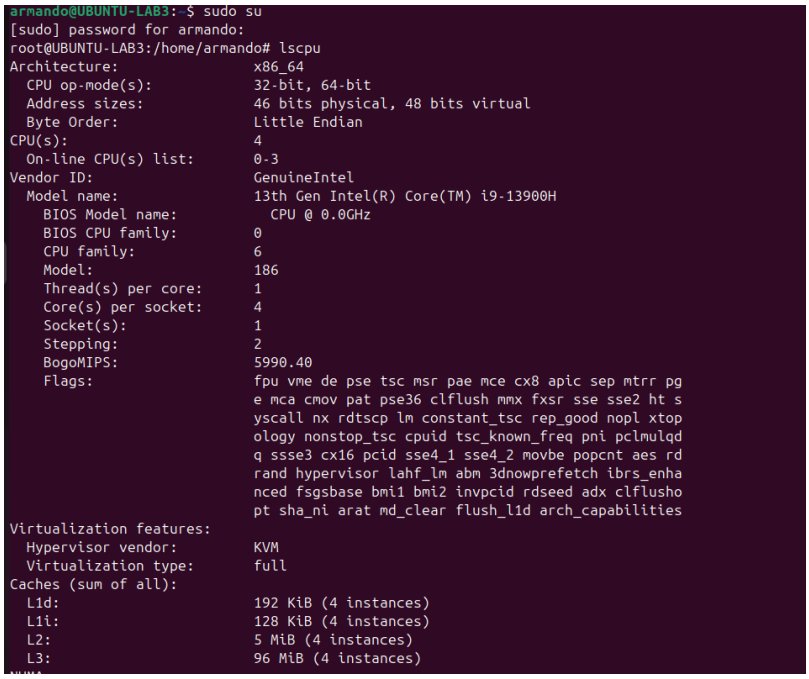


Parte 1: Exploración de la Arquitectura del Sistema

Comandos de verificación

lscpu

Ilustración 3 Captura de VM , Resultado lscpu



```
armando@UBUNTU-LAB3:~$ sudo su
[sudo] password for armando:
root@UBUNTU-LAB3:/home/armando# lscpu
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 46 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          4
  On-line CPU(s) list:           0-3
Vendor ID:                       GenuineIntel
  Model name:                    13th Gen Intel(R) Core(TM) i9-13900H
    BIOS Model name:               CPU @ 0.0GHz
    BIOS CPU family:             0
    CPU family:                  6
    Model:                       186
    Thread(s) per core:          1
    Core(s) per socket:          4
    Socket(s):                   1
    Stepping:                    2
    BogoMIPS:                    5990.40
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                                 e mca cmov pat pse36 clflush mmx fxsr sse sse2 ht s
                                 yscall nx rdtscp lm constant_tsc rep_good nopl xtop
                                 ology nonstop_tsc cpuid tsc_known_freq pni pclmulqd
                                 q ssse3 cx16 pcid sse4_1 sse4_2 movbe popcnt aes rd
                                 rand hypervisor lahf_lm abm 3dnowprefetch ibrs_enha
                                 nced fsgsbase bmi1 bmi2 invpcid rdseed adx clflusho
                                 pt sha_ni arat md_clear flush_l1d arch_capabilities
Virtualization features:
  Hypervisor vendor:             KVM
  Virtualization type:           full
Caches (sum of all):
  L1d:                           192 KiB (4 instances)
  L1i:                           128 KiB (4 instances)
  L2:                            5 MiB (4 instances)
  L3:                            96 MiB (4 instances)
```

free -h

Ilustración 4 Captura de VM , Resultado free -h

```
root@UBUNTU-LAB3:/home/armando# free -h
               total        used        free      shared  buff/cache   available
Mem:           7.8Gi       2.9Gi       1.0Gi        81Mi       4.2Gi       4.9Gi
Swap:             0B          0B          0B
```

lscpu | grep cache

Ilustración 5 Captura de VM , Resultado lscpu grep cache

```
root@UBUNTU-LAB3:/home/armando# lscpu | grep cache
L1d cache:                               192 KiB (4 instances)
L1i cache:                               128 KiB (4 instances)
L2 cache:                                5 MiB (4 instances)
L3 cache:                                96 MiB (4 instances)
```

uname -a

Ilustración 6 Captura de VM , Resultado uname -a

```
root@UBUNTU-LAB3:/home/armando# uname -a
Linux UBUNTU-LAB3 6.17.0-19-generic #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 23:08:46 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

dmidecode -t memory

Ilustración 7 Captura de VM , Resultado dmidecode -t memory

```
root@UBUNTU-LAB3:/home/armando# sudo dmidecode -t memory
# dmidecode 3.5
Getting SMBIOS data from sysfs.
SMBIOS 2.5 present.
```

Resultado de la Verificación:

1. **¿Cuántos núcleos tiene el CPU?**

   Tiene 4 núcleos visibles en la VM.

2. **¿Cuál es el tamaño de la cache L1?**

La caché L1 aparece separada en:

L1d: 192 KiB (4 instancias)

L1i: 128 KiB (4 instancias)

3. ¿Cuál es el tamaño de la cache L2?

5 MiB

4. ¿Cuál es el tamaño de la cache L3?

96 MiB

5. ¿Qué arquitectura utiliza el sistema?

x86_64, es decir, arquitectura de 64 bits.

Parte 2: Benchmark de Latencia de Memoria

Se evaluó la latencia de acceso a memoria utilizando la herramienta lmbench, con el objetivo de observar cómo varía el tiempo de acceso cuando cambia el tamaño del working set. Estas mediciones permiten identificar el efecto de la jerarquía de memoria del sistema, evidenciando la diferencia de rendimiento entre los distintos niveles de caché y la memoria principal.

Ilustración 8 lat_mem_rd 4K

```
root@UBUNTU-LAB3:/home/armando# /usr/lib/lmbench/bin/x86_64-linux-gnu/lat_mem_rd 4K 128
"stride=128
0.00049 0.968
0.00098 0.959
0.00195 0.969
0.00293 1.096
0.00391 1.081
0.00586 0.979
0.00781 1.090
0.01172 0.965
0.01562 1.133
0.02344 0.968
0.03125 1.005
0.04688 1.132
0.06250 1.828
0.09375 2.299
0.12500 1.652
0.18750 1.627
0.25000 1.776
0.37500 1.847
0.50000 1.525
0.75000 1.777
1.00000 1.742
1.50000 2.331
2.00000 2.753
3.00000 3.723
4.00000 3.481
```

Ilustración 9 lat_mem_rd 32K

```
root@UBUNTU-LAB3:/home/armando# /usr/lib/lmbench/bin/x86_64-linux-gnu/lat_mem_rd 32K 128
"stride=128
0.00049 1.045
0.00098 1.058
0.00195 1.111
0.00293 1.126
0.00391 1.117
0.00586 1.139
0.00781 1.118
0.01172 0.980
0.01562 1.110
0.02344 0.986
0.03125 1.193
0.04688 1.152
0.06250 2.397
0.09375 2.055
0.12500 1.935
0.18750 2.063
0.25000 1.950
0.37500 2.978
0.50000 2.692
0.75000 2.471
1.00000 2.873
1.50000 2.676
2.00000 3.897
3.00000 2.332
4.00000 2.378
6.00000 2.354
8.00000 3.399
12.00000 3.990
16.00000 3.753
24.00000 6.152
32.00000 6.997
```

Ilustración 10 lat_mem_rd 256K

```
root@UBUNTU-LAB3:/home/armando# /usr/lib/lmbench/bin/x86_64-linux-gnu/lat_mem_rd 256K 128
"stride=128
0.00049 1.070
0.00098 1.085
0.00195 1.076
0.00293 1.078
0.00391 1.048
0.00586 0.957
0.00781 1.074
0.01172 0.966
0.01562 0.953
0.02344 1.138
0.03125 0.975
0.04688 1.135
0.06250 2.625
0.09375 1.787
0.12500 1.783
0.18750 1.512
0.25000 1.968
0.37500 1.767
0.50000 2.293
0.75000 3.096
1.00000 1.670
1.50000 2.045
2.00000 2.168
3.00000 1.969
4.00000 2.761
6.00000 3.086
8.00000 2.924
12.00000 3.973
16.00000 3.892
24.00000 6.219
32.00000 6.965
48.00000 8.809
64.00000 7.594
96.00000 7.764
128.00000 7.669
192.00000 6.785
256.00000 7.972
```

Ilustración 11 lat_mem_rd 8M

```
root@UBUNTU-LAB3:/home/armando# /usr/lib/lmbench/bin/x86_64-linux-gnu/lat_mem_rd 8M 128
"stride=128
0.00049 1.069
0.00098 0.969
0.00195 0.957
0.00293 0.954
0.00391 0.978
0.00586 1.012
0.00781 1.119
0.01172 1.109
0.01562 1.078
0.02344 1.079
0.03125 1.084
0.04688 1.089
0.06250 2.646
0.09375 2.675
0.12500 1.819
0.18750 1.638
0.25000 1.903
0.37500 1.784
0.50000 1.697
0.75000 2.269
1.00000 2.006
1.50000 2.316
2.00000 2.096
3.00000 2.163
4.00000 2.240
6.00000 2.506
8.00000 2.516
```

Ilustración 12 lat_mem_rd 64M

```
root@UBUNTU-LAB3:/home/armando# /usr/lib/lmbench/bin/x86_64-linux-gnu/lat_mem_rd 64M 128
"stride=128
0.00049 1.006
0.00098 0.995
0.00195 1.126
0.00293 1.132
0.00391 1.126
0.00586 1.103
0.00781 1.029
0.01172 1.123
0.01562 1.007
0.02344 1.135
0.03125 1.127
0.04688 1.018
0.06250 2.289
0.09375 2.624
0.12500 2.369
0.18750 1.966
0.25000 2.306
0.37500 1.412
0.50000 2.012
0.75000 2.546
1.00000 2.313
1.50000 2.513
2.00000 2.791
3.00000 2.442
4.00000 2.491
6.00000 3.088
8.00000 3.471
12.00000 5.438
16.00000 3.501
24.00000 5.036
32.00000 6.619
48.00000 7.338
64.00000 8.493
```

Tabla de resultados

Tabla 1 Resultados Working Set Latency

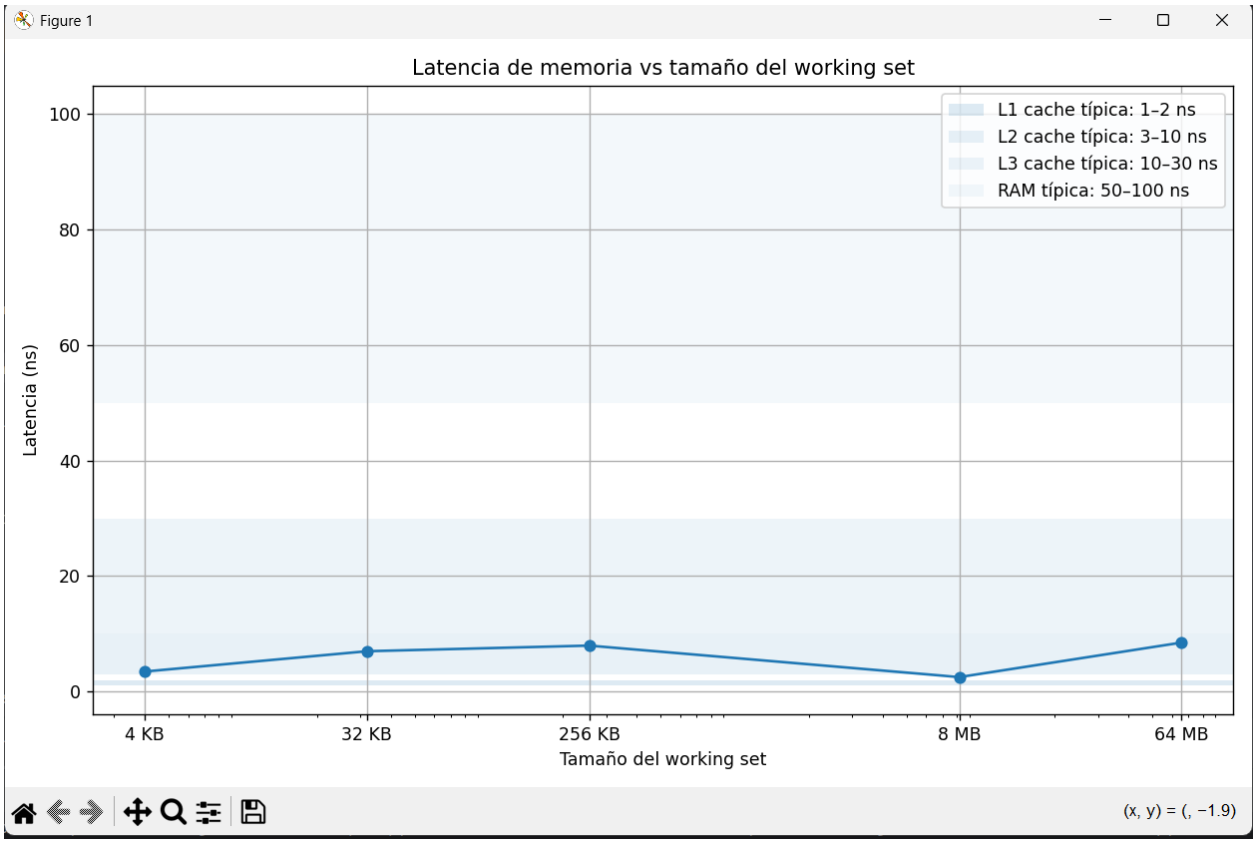
Working Set	Latency (ns)
4 KB	3.481 ns
32 KB	6.997 ns
256 KB	7.972 ns
8 MB	2.516 ns

64 MB	8.493 ns

Parte 3: Visualización de la Jerarquía de Memoria

En esta sección se representaron gráficamente los resultados obtenidos en el benchmark de latencia, relacionando el tamaño del working set con la latencia de acceso. La visualización permite identificar de forma más clara el comportamiento de la jerarquía de memoria, mostrando cómo la latencia cambia cuando el conjunto de trabajo supera la capacidad de los distintos niveles de caché.

Ilustración 13 Latencia de Memoria vs Tamaño del Working set



Preguntas y respuestas:

1. ¿En qué puntos de la gráfica se observan transiciones entre niveles de cache?

Las transiciones más visibles se observan cuando la latencia cambia de forma notable entre un tamaño y otro del working set. En tu gráfica, eso pasa principalmente:

- de 4 KB a 32 KB, donde la latencia sube,
- de 32 KB a 256 KB, donde vuelve a subir,
- y de 8 MB a 64 MB, donde hay otro incremento claro.

Eso sugiere que, al crecer el working set, los datos dejan de caber completamente en un nivel de caché más rápido y empiezan a depender de otro nivel más lento.

2. ¿Cómo se relaciona esto con la jerarquía de memoria del sistema?

Se relaciona directamente con la jerarquía de memoria:

- L1 es la más rápida y pequeña,
- L2 es más grande pero un poco más lenta,
- L3 es aún más grande y más lenta,
- y la RAM es mucho más grande, pero con mayor latencia.

Cuando el working set es pequeño, el procesador puede mantener la mayoría de los datos en cachés rápidas. Pero cuando el conjunto de trabajo crece y supera la capacidad de esas cachés, tiene que buscar datos en niveles más alejados del CPU, lo que aumenta el tiempo de acceso.

3. ¿Por qué aumenta la latencia cuando el working set supera el tamaño de cache?

Porque ya no todos los datos caben en la caché correspondiente. Entonces ocurren más fallos de caché, y el procesador debe traer información desde un nivel más lento de la jerarquía de memoria.

Parte 4: Experimento de Comportamiento de Cache

En esta parte del laboratorio se implementó un programa C sencillo para analizar el comportamiento de la memoria caché mediante la comparación de dos patrones de acceso: secuencial y aleatorio. El objetivo fue observar cómo la forma en que un programa accede a los datos influye directamente en el rendimiento, debido al aprovechamiento o no de la localidad de referencia y de los mecanismos internos de la caché.

Código del Programa:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 10000000
#define ITERATIONS 5

double diff_in_ms(struct timespec start, struct timespec end) {
   return (end.tv_sec - start.tv_sec) * 1000.0 +
         (end.tv_nsec - start.tv_nsec) / 1000000.0;
}

int main() {
   int *array = malloc(SIZE * sizeof(int));
   if (array == NULL) {
```

```
        printf("Error al asignar memoria\n");
        return 1;
    }

    for (int i = 0; i < SIZE; i++) {
        array[i] = 0;
    }

    struct timespec start, end;
    double sequential_time = 0.0;
    double random_time = 0.0;

    srand(time(NULL));

    for (int k = 0; k < ITERATIONS; k++) {
        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 0; i < SIZE; i++) {
            array[i]++;
        }

        clock_gettime(CLOCK_MONOTONIC, &end);
        sequential_time += diff_in_ms(start, end);
    }

    for (int k = 0; k < ITERATIONS; k++) {
        clock_gettime(CLOCK_MONOTONIC, &start);

        for (int i = 0; i < SIZE; i++) {
            int index = rand() % SIZE;
            array[index]++;
        }

        clock_gettime(CLOCK_MONOTONIC, &end);
        random_time += diff_in_ms(start, end);
    }

    printf("Promedio acceso secuencial: %.3f ms\n", sequential_time / ITERATIONS);
    printf("Promedio acceso aleatorio: %.3f ms\n", random_time / ITERATIONS);

    free(array);
    return 0;
}
```

Procedimiento de ceración y ejecución en Linux:

- nano cache_test.c
- gcc cache_test.c -o cache_test
- ./cache_test
- sudo apt update
- sudo apt install -y build-essential
- gcc cache_test.c -o cache_test
- ./cache_test

Verificación Visual en la máquina Virtual:

Ilustración 14 Resultados en VM Ejecutando el Programa cache.test

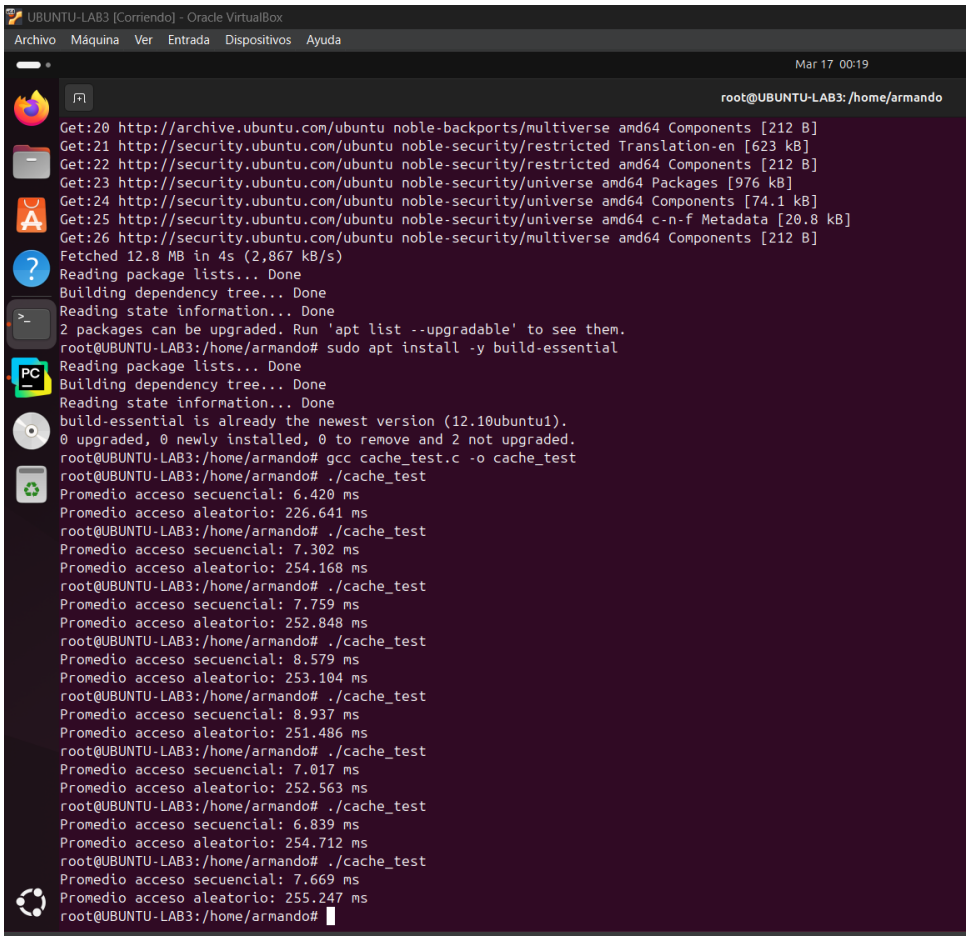


Tabla de Resultados:

Tabla 2 Resultado ejecución VM Programa

Ejecución	Secuencial (ms)	Aleatorio (ms)
1	6.420	226.641
2	7.302	254.168
3	7.759	252.848
4	8.579	253.104
5	8.937	251.486
6	7.017	252.563
7	6.839	254.712
8	7.669	255.247
Promedio	**7.565**	**250.096**

Preguntas y Respuestas:

1. ¿Cuál patrón fue más rápido?

El patrón de acceso secuencial fue mucho más rápido que el acceso aleatorio. En las pruebas realizadas, el promedio del acceso secuencial fue de aproximadamente 7.565 ms, mientras que el acceso aleatorio fue de 250.096 ms.

2. ¿Por qué el acceso secuencial es más eficiente?

El acceso secuencial es más eficiente porque recorre posiciones contiguas en memoria. Esto permite que el procesador aproveche mejor la caché, ya que cuando se carga una dirección de memoria, también se cargan datos cercanos. Como el siguiente acceso probablemente estará en una posición adyacente, muchos datos ya estarán disponibles en caché, reduciendo el tiempo de acceso.

3. Explique el papel de:

Locality of reference

La localidad de referencia es el principio según el cual un programa tiende a acceder repetidamente a los mismos datos o a datos cercanos entre sí. En el acceso secuencial se aprovecha especialmente la localidad espacial, porque se recorren posiciones vecinas del arreglo. Esto mejora el rendimiento al reducir fallos de caché.

Cache lines

La memoria caché no trae un solo dato aislado, sino un bloque llamado cache line. Cuando el programa accede a una posición del arreglo, también se cargan en caché varios datos contiguos. En el acceso secuencial, esto es muy beneficioso porque los siguientes elementos ya pueden venir dentro de la misma línea de caché.

Hardware prefetching

El hardware prefetching es un mecanismo del procesador que intenta anticipar qué datos serán usados después y los carga en caché antes de que el programa los solicite. En accesos secuenciales funciona muy bien, porque el patrón es predecible. En cambio, en accesos aleatorios casi no ayuda, ya que los siguientes accesos no siguen un orden claro. (Drepper, 2007)

Parte 5: Implicaciones de Seguridad

Meltdown es un ataque de ejecución especulativa que aprovecha una optimización de rendimiento presente en procesadores modernos. Su funcionamiento se basa en que el CPU puede ejecutar instrucciones de forma adelantada antes de confirmar si realmente están permitidas. En este proceso, un programa sin privilegios puede intentar leer una dirección de memoria del kernel. Aunque arquitectónicamente ese acceso debería ser bloqueado y luego descartado, durante la ejecución especulativa algunos efectos microarquitectónicos sí ocurren, especialmente en la caché. (Lipp, y otros, 2018)

El ataque no obtiene el dato de manera directa visible al programa, sino de forma indirecta. El valor leído especulativamente se usa para acceder a posiciones específicas de memoria controladas por el atacante. Ese acceso deja huellas en la caché. Luego, midiendo tiempos de acceso a distintas posiciones, el atacante puede inferir cuál dato fue cargado, porque las posiciones que quedaron en caché responden más rápido.

La jerarquía de memoria permite este tipo de ataques porque introduce diferencias medibles de tiempo entre acceder a datos en L1, L2, L3 o RAM. Esas diferencias, aunque fueron diseñadas para mejorar rendimiento, también crean un canal lateral. En otras palabras, aunque el

sistema bloquee el acceso a nivel lógico, los cambios en la caché revelan información sobre lo que ocurrió internamente en el procesador.

Por eso, la jerarquía de memoria se vuelve clave en estos ataques: la caché no solo acelera la ejecución, sino que también deja rastros observables. Meltdown demuestra que una optimización pensada para velocidad puede transformarse en una vulnerabilidad cuando los efectos micro arquitectónicos filtran información sensible. (Drepper, 2007)

Relación con el Laboratorio en el ámbito Practico:

La relación con la parte práctica del laboratorio está en que los experimentos realizados muestran que el tiempo de acceso a memoria cambia según si los datos están en caché o en memoria principal, y también según el patrón de acceso utilizado. En el benchmark con lmbench se observó cómo la latencia varía al aumentar el working set, evidenciando la existencia de distintos niveles de caché. Asimismo, en el experimento de acceso secuencial y aleatorio se comprobó que la caché mejora notablemente el rendimiento cuando existe localidad de referencia.

Esta misma diferencia de tiempos es la base de ataques como Meltdown. Aunque el acceso indebido a memoria privilegiada sea bloqueado a nivel arquitectónico, durante la ejecución especulativa el procesador puede dejar rastros en la caché. Luego, midiendo tiempos de

acceso, el atacante puede inferir qué datos fueron tocados. Es decir, el laboratorio demuestra en la práctica que la jerarquía de memoria no solo mejora rendimiento, sino que también puede convertirse en un canal lateral de información.

En otras palabras, los experimentos del laboratorio permiten entender por qué la caché es tan importante en seguridad: las diferencias de latencia que nosotros usamos para medir rendimiento son las mismas que un atacante puede usar para extraer información secreta.

Conclusiones

A través de este laboratorio se pudo comprobar que la jerarquía de memoria tiene un impacto fundamental en el desempeño de un sistema computacional. Los experimentos mostraron que la latencia de acceso aumenta conforme el working set supera la capacidad de los niveles de caché, y que el patrón de acceso a memoria influye significativamente en el tiempo de ejecución. En particular, el acceso secuencial resultó mucho más eficiente que el acceso aleatorio, debido a que aprovecha mejor la localidad espacial, las líneas de caché y los mecanismos de prefetching del hardware.

Además, el laboratorio permitió entender que la caché no solo es un mecanismo de optimización de rendimiento, sino también un elemento

clave para comprender el comportamiento real de los programas y, en ciertos casos, incluso vulnerabilidades de seguridad basadas en diferencias de tiempo. En general, la práctica ayudó a reforzar de manera clara la relación entre arquitectura, rendimiento y organización de memoria en sistemas modernos.

Referencias

Drepper, U. (2007). *What Every Programmer Should Know About Memory.*

Hennessy, J. L., & Patterson, D. A. . (2012). Computer architecture: A quantitative approach.

Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., . . . Hamburg, M. (2018). *Meltdown: Reading Kernel Memory from User Space.* USENIX Association.

Patterson, D.A, & Hennessy, J.L. (2014). Computer organization and design: The hardware/software interface.